

APISAVE V10.1 - REFUGE INTÉGRAL DE PRODUCTION (CODE ET SPECS)

Ce document constitue l'unique source de vérité (Master Doc) regroupant à l'identique l'intégralité absolue du code source V12, l'architecture hors-ligne, le guide d'intégration, ainsi que toutes les modifications fonctionnelles requises par les spécifications terrain de 2026 (Refonte Accueil, Barre de Navigation sans émoji, et Validation Critique du GPS EXIF).

SECTION 1: ARCHITECTURE TECHNIQUE BACKEND & MOTEUR JUGE

1.1 - constants/verdicts.js

```
// constants/verdicts.js
// APISAVE BEEALERT CORE V13.5+ MES-1 V3 terrain correctif V3.5+

export const REASON_CODES = {
  NONE: 'NONE',
  'RETAKE DORSAL VIEW': 'RETAKE DORSAL VIEW',
  'RETAKE THORAX': 'RETAKE THORAX',
  'RETAKE ABDOMEN': 'RETAKE ABDOMEN',
  'RETAKE MORPHOLOGY': 'RETAKE MORPHOLOGY',
  RETAKE_SHARPER: 'RETAKE_SHARPER',
  RETAKE_LIGHTING_ANGLE: 'RETAKE_LIGHTING_ANGLE',
  'RETAKE PROFILE': 'RETAKE PROFILE',
  'RETAKE INSECT': 'RETAKE INSECT',
  'STRUCTURE STRONG GLOBAL': 'STRUCTURE STRONG GLOBAL',
  'STRUCTURE MEDIUM GLOBAL': 'STRUCTURE MEDIUM GLOBAL',
  'INSECT TOO BLURRY': 'INSECT TOO BLURRY',
  NO_CRITERIA_VISIBLE: 'NO_CRITERIA_VISIBLE',
  'OBJECT NON BIOLOGICAL STRUCTURE': 'OBJECT_NON BIOLOGICAL STRUCTURE',
  'INSECT HAIRY BODY INCOMPATIBLE': 'INSECT HAIRY BODY INCOMPATIBLE',
  'INSECT BEETLE FEATURES VISIBLE': 'INSECT BEETLE FEATURES VISIBLE',
  'CRABRO_LIKE_PROFILE': 'CRABRO_LIKE_PROFILE'
};

export const CONFIANCE = {
```

```

    ROUGE: 92,
    ORANGE_PLAFOND: 72,
    ORANGE_PROBABLE_NON_CIBLE: 65,
    ORANGE_INSUFFISANCE: 55,
    VERT: 85
  };

  export const ACTIONS = {
    ROUGE: ['CONTACT PRO', 'SIGNAL'],
    ORANGE_PLAFOND: ['CONTACT PRO', 'SIGNAL'],
    ORANGE_PROBABLE_NON_CIBLE: ['NO ACTION'],
    ORANGE_INSUFFISANCE: ['RETAKE'],
    VERT: ['NO_ACTION']
  };

```

1.2 - constants/branding.js

```

// constants/branding.js
// APISAVE BEEALERT CORE V13.5+ MES-1 Version V3.5+

export const ENGINE = {
  protocole: 'V13.5+MES-1+V3.5+',
  version_juge: '1.8.3+V3.5+',
  version_vision: '2.1+V3.5+'
};

```

1.3 - apisave-prompt-vision.js

```

// apisave-prompt-vision.js
// APISAVE PROMPT VISION V2.1+V3.5+ (Version de Production durcie, zéro variabilité)
// Compatible BEEALERT CORE V13.5+ MES-1 Version V10.1 Production Terrain
Integrated

export const VISION_SYSTEM_PROMPT_APISAVE = `Tu es un observateur visuel
entomologique strict.
Tu décris UNIQUEMENT ce qui est clairement visible sur l'image.
Tu dois formuler ton analyse de manière séquentielle, logique et déterministe en remplissant

```

le JSON final etape par etape.

<INTERDICTIONS ABSOLUES>

Ne JAMAIS nommer d'espece.

Ne JAMAIS utiliser de termes de verdict (rouge, orange, danger, alerte, suspicion, etc.).

Ne jamais extrapoler, supposer, deviner ou completer. Aucun critere invente.

Si several individus sont presents: choisir UN SEUL individu, le plus net et le plus exploitable.

TOUTES les observations Q1, Q2, Q3 doivent porter EXCLUSIVEMENT sur lui.

Interdiction absolue de melanger des criteres provenant d'individus differents.

</INTERDICTIONS ABSOLUES>

<PIPELINE ANALYSE DETERMINISTE>

Tu DOIS executer les blocs suivants dans l'ordre strict.

=== ETAPE 1: VERROUX STRUCTURES ET PIEGES (MODE FAIL-FAST) ===

SI (Aucun insecte exploitable n'est visible) ALORS:

Symetrie parfaite, repetition reguliere, surface plastique/lisse, armature/cable/lanterne visible -> Ajouter 'OBJECT_NON BIOLOGICAL_STRUCTURE'.

Ne jamais valider comme nid.

Objet sur branche/pin avec texture soyeuse, cocon, amas vegetal/feuilles, galle ou fruit ->

Traiter comme piege vegetal.

Interdiction de valider une stratification cartonnee.

=== ETAPE 2: GESTION DES PERTURBATEURS INSECTE ===

SI (Insecte exploitable visible) ALORS:

IGNORER (ne declenche pas NON LISIBLE): miel, liquide, piege a appat, surface reflechissante, gouttes, eclairage non uniforme.

BLINDAGE ANTI-CONTAMINATION CHROMATIQUE: Si l'insecte est pose sur brique rouge, tuile orange, bois roux ou plastique colore, ignorer les reflets roux/chauds.

Juger UNIQUEMENT la couleur reelle du tegument (noir ou brun).

=== ETAPE 3: VERROUX D'EXCLUSION INSECTE ===

SI (Insecte exploitable visible) ALORS:

VERROU GUEPE/POLISTE/HYMENOPTERE STRICT:

SI l'insecte presente AU MOINS UN des marqueurs suivants :

- Silhouette globalement fine, elancee, svelte ou filiforme.

- Pattes majoritairement jaunes/rousses/clairer DE LA BASE AUX EXTREMITES sans rupture noire epaisse et massive pres du corps.

- Motif de rayures transversales jaunes/clair (valable par regle anti-artefact triple courbure naturelle repetition sur ≥ 2 segments + homogeneite).

- Motif regulier alterne jaune/noir (lignes continues, chevrons ou triangles repetes sur fond jaune dominant).

ALORS:

-> FORCER Q2 abdomen.reponse = 'NON'

-> FORCER Q3 morphologie.reponse = 'NON'

-> Ajouter dans incompatibilites_cible: 'silhouette_fine_allongee',
'proportions_greles_non_robustes', 'rayures_jaune_noir_vif', 'abdomen_jaune_dominant'

-> STOP_EVALUATION_CIBLE

VERROU BOURDON/COLEOPTERE/MICRO:

Corps tres poilu/massif -> Q3 = 'NON' + 'INSECT HAIRY BODY INCOMPATIBLE'

Carapace dure/elytres visibles -> Q3 = 'NON' + 'INSECT BEETLE FEATURES VISIBLE'

Taille minuscule par rapport au support -> Q3 = 'NON' +

'insecte_taille_minuscule_non_frelon'

=== ETAPE 4: EVALUATION CIBLE (SI AUCUN VERROU N'A BLOQUE) ===

<Q1_THORAX>

OUI = Couleur de fond majoritairement noire ou brun tres fonce.

NON = Fond majoritairement roux, brun-roux, rougeatre ou tete rousse dominante.

NON LISIBLE = Thorax invisible, masque, trop flou ou vue ventrale non exploitable.

</Q1_THORAX>

<Q2_ABDOMEN>

OUI uniquement si TOUTES les conditions sont reunies:

1. Fond dominant sombre (noire ou noire-brune sur les 3/4 superieurs).
2. Zone orangee/jaune-orange fonce presente UNIQUEMENT vers l'extremite (4eme segment).
3. ABSENCE TOTALE de lignes claires continues traversant horizontalement le dos.

REGLE ANTI-ARTEFACT TRIPLE (V3.5+):

Une ligne claire n'est consideree comme motif de guepe que si elle remplit simultanement:

Courbure: suit la courbure naturelle des plaques abdominales.

Repetition: presente sur > 2 segments consecutifs.

Homogeneite: epaisseur et continuite regulieres.

Ignorer absolument: lignes droites, isolees, parasites, reflets, ombres, grillage, brindilles.

NOTE: Tres fins liseres intersegmentaires discontinus n'invalident pas Q2 OUI si le fond reste sombre.

</Q2_ABDOMEN>

<Q3_MORPHOLOGIE>

OUI = Au moins 2 elements: thorax massif, jonction thorax/abdomen large, abdomen epais non elance, proportions compactes robustes.

NON = Morphologie clairement grele, filiforme, jonction etroite, silhouette tres fine.

</Q3_MORPHOLOGIE>

=== ETAPE 5: EVALUATION STRUCTURE (UNIQUEMENT SI PAS D'INSECTE EXPLOITABLE) ===

texture_papier_carton OUI si surface mate fibreuse grise/beige cartonnee homogene travaillee (NON si vegetal, mousse, lichen, soie).

strates_repetitives OUI si couches ou ondulations paralleles visibles.

REGLE IMPORTANTE: texture_papier_carton + strates_repetitives ne suffit JAMAIS seul.

Il faut une organisation construite repetitive avec irregularite organique compatible.

structure_strength = STRONG (construite, coherente) | MEDIUM | WEAK (doute, ambigu, confusion possible).

</PIPELINE ANALYSE DETERMINISTE>

<FORMAT_DE_SORTIE_OBLIGATOIRE>

Reponds UNIQUEMENT avec un objet JSON valide. Aucun texte avant, aucun texte apres.

</FORMAT_DE_SORTIE_OBLIGATOIRE>`;

export const VISION_USER_PROMPT = `Analyse cette image en appliquant strictement les regles du prompt systeme.

Reponds UNIQUEMENT avec le JSON suivant :

```
{
  "etape_1_declencheur": { "insecte_exploitable": true, "structure_visible": false,
    "justification": "... " },
  "etape_2_individu": { "individu_analyse_identifiable": true, "vue_dorsale": true,
    "sur_le_dos": false },
  "Q1_thorax": { "reponse": "OUI|NON|NON_LISIBLE", "confidence": "LOW|MEDIUM|HIGH",
    "description_visible": "...", "lisibilite": "haute|moyenne|non_lisible" },
  "Q2_abdomen": { "reponse": "OUI|NON|NON_LISIBLE", "confidence":
    "LOW|MEDIUM|HIGH", "fond_dominant":
    "sombre|jaune_clair|jaune_vif|orange|mixte_jaune_noir_alterne|non_lisible",
    "zone_terminale_orangee": true, "description_visible": "...", "lisibilite":
```

```

"haute|moyenne|non_lisible" },
  "Q3_morphologie": { "reponse": "OUI|NON|NON_LISIBLE", "confidence":
"LOW|MEDIUM|HIGH", "elements_visibles": [], "incompatibilites_visibles": [],
"description_visible": "...", "lisibilite": "haute|moyenne|non_lisible" },
  "incompatibilites_cible": [],
  "structure": {
    "evaluee": false,
    "forme_globale": "ovoide|spherique|irreguliere|aplatie|non_lisible",
    "texture_papier_carton": "OUI|NON|NON_LISIBLE",
    "strates_repetitives": "OUI|NON|NON_LISIBLE",
    "suspension_visible": "OUI|NON|NON_LISIBLE",
    "position": "arbre|toiture|haie|sol|cavite|support_artificiel|non_lisible",
    "qualite_structure": "LOW|MEDIUM|HIGH",
    "structure_strength": "STRONG|MEDIUM|WEAK",
    "marqueurs_forts": [],
    "marqueurs_faibles": [],
    "indices_artificiels": [],
    "pieges_vegetaux_possibles": []
  }
};

```

1.4 - apisave-juge.js

```

// apisave-juge.js
// APISAVE MOTEUR DE DECISION (LE JUGE) BEEALERT CORE V13.5+ MES-1
// Version 1.8.3+V3.5+ terrain (Production Clean Coupe-Circuit + Patch texture)

import { CONFIANCE, ACTIONS, REASON_CODES } from './constants/verdicts.js';
import { ENGINE } from './constants/branding.js';

const ANTI_CRABRO_TYPES = new Set([
  'abdomen jaune dominant', 'rayures jaune noir_vif',
  'abdomen_segmente_jaune_noir_alterne', 'thorax_roux',
  'tete_rousse_orangee'
]);

const MORPHO_TYPES = new Set([
  'morphologie_filiforme', 'silhouette tres_fine',

```

```

'morphologie_valeur_compacte',
'jonction_etroite', 'proportions greles_non robustes',
'silhouette_fine_allongee',
'insecte_taille_minuscule_non_frelon',
'INSECT HAIRY BODY INCOMPATIBLE'
]);

```

```

const EMOJI_MAP = {
  ROUGE: 'rouge',
  ORANGE_PLAFOND: 'orange_plafond',
  ORANGE_PROBABLE_NON_CIBLE: 'orange_probable_non_cible',
  ORANGE_INSUFFISANCE: 'orange_insuffisance',
  VERT: 'vert'
};

```

```

function generateId() {
  return 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, c => {
    const r = Math.random() * 16 | 0;
    const v = c === 'x' ? r : (r & 0x3 | 0x8);
    return v.toString(16);
  });
}

```

```

function formatVerdict(verdictKey, motif, reasonCode, analyseld, timestamp) {
  if (!(reasonCode in REASON_CODES)) throw new Error(`reason_code inconnu: \${reasonCode}`);
  return {
    analyse_id: analyseld,
    timestamp,
    verdict: EMOJI_MAP[verdictKey],
    verdict_code: verdictKey,
    confiance: CONFIANCE[verdictKey],
    motif_principal: motif,
    reason_code: reasonCode,
    actions_recommandees: ACTIONS[verdictKey],
    avis: 'Analyse indicative basee uniquement sur les elements visibles. Confirmation professionnelle recommandee.',
    protocole_version: ENGINE.prototype,
  };
}

```

```
}
```

```
function effectiveReponse(reponse, confidence) {  
  if (reponse === 'NON' && confidence === 'LOW') return 'NON LISIBLE';  
  return response;  
}
```

```
function calibrateConfiance(baseConfiance, q1conf, q2conf, q3conf) {  
  const confs = [q1conf, q2conf, q3conf].filter(Boolean);  
  if (confs.length == 0) return baseConfiance;  
  const highCount = confs.filter(c => c === 'HIGH').length;  
  const lowCount = confs.filter(c => c === 'LOW').length;  
  let adjusted = baseConfiance;  
  if (highCount === 3) adjusted = Math.min(adjusted + 5, 98);  
  else if (lowCount >= 2) adjusted = Math.max(adjusted - 15, 30);  
  else if (lowCount === 1) adjusted = Math.max(adjusted - 8, 35);  
  return adjusted;  
}
```

```
function verrouvert(obs, q1, q2, q3, surLeDos, analyseld, timestamp, incompat, nbMorpho,  
nbTotal, antiCrabroHit) {  
  if (q1 === 'NON' && q2 === 'NON') {  
    if (nbTotal >= 3 && nbMorpho >= 1) {  
      return formatVerdict('VERT', 'Profil fortement incompatible avec Vespa velutina  
(multiples incompatibilites).', 'NONE', analyseld, timestamp);  
    }  
  }  
  const isCertitudeHaute = obs?.Q1_thorax?.confidence === 'HIGH' &&  
obs?.Q2_abdomen?.confidence === 'HIGH';  
  if ((q3 === 'OUI' || q3 === 'NON LISIBLE') && q1 === 'NON' && q2 === 'NON' &&  
nbMorpho === 0) {  
    if (antiCrabroHit >= 3 || (antiCrabroHit === 2 && isCertitudeHaute)) {  
      return formatVerdict('ORANGE PROBABLE NON CIBLE', 'Convergence claire de  
marqueurs type crabro avec morphologie frelon validee.', 'CRABRO_LIKE_PROFILE',  
analyseld, timestamp);  
    }  
  }  
  if (nbMorpho >= 2) {  
    return formatVerdict('VERT', '>= 2 incompatibilites morphologiques claires avec Vespa
```



```

velutina.', 'NONE', analyseld, timestamp);
}
if (antiCrabroHit >= 2) {
  const reason = surLeDos ? 'RETAKE DORSAL VIEW' : 'RETAKE LIGHTING ANGLE';
  return formatVerdict('ORANGE_INSUFFISANCE', 'Profil chromatique fortement
crabro-like seconde vue requise.', reason, analyseld, timestamp);
}
if (q3 === 'OUI' || q3 === 'NON_LISIBLE') {
  const reason = surLeDos ? 'RETAKE DORSAL VIEW' : 'RETAKE PROFILE';
  return formatVerdict('ORANGE_INSUFFISANCE', 'Morphologie compatible frelon, criteres
chromatiques insuffisants seconde photo.', reason, analyseld, timestamp);
}
if (q3 === 'NON') {
  if (nbMorpho === 0 && antiCrabroHit >= 1 && nbTotal < 3) {
    const reason = surLeDos ? 'RETAKE DORSAL VIEW' : 'RETAKE_LIGHTING_ANGLE';
    return formatVerdict('ORANGE_INSUFFISANCE', 'Morphologie deviante mais profil
crabro seconde photo requise.', reason, analyseld, timestamp);
  }
  if (nbTotal >= 2) {
    return formatVerdict('VERT', '>= 2 incompatibilites claires avec Vespa velutina.', 'NONE',
analyseld, timestamp);
  }
}
return formatVerdict('ORANGE INSUFFISANCE', 'Criteres insuffisants pour conclure.',
'RETAKE SHARPER', analyseld, timestamp);
}

```

```

function jugerMorphologie(obs, analyseld, timestamp) {
  const surLeDos = obs.etape_2_individu.sur_le_dos;
  const q1conf = obs.Q1_thorax.confidence || 'MEDIUM';
  const q2conf = obs.Q2_abdomen.confidence || 'MEDIUM';
  const q3conf = obs.Q3_morphologie.confidence || 'MEDIUM';

  let q1 = effectiveReponse(obs.Q1_thorax.reponse, q1conf);
  let q2 = effectiveReponse(obs.Q2_abdomen.reponse, q2conf);
  let q3 = effectiveReponse(obs.Q3_morphologie.reponse, q3conf);

  if (obs.Q1_thorax.lisibilite === 'non_lisible' || q1 === 'NON LISIBLE') q1 = 'NON LISIBLE';
  if (obs.Q2_abdomen.lisibilite === 'non_lisible' || q2 === 'NON LISIBLE') q2 =

```

```

'NON_LISIBLE';
  if (obs.Q3_morphologie.lisibilite === 'non_lisible' || q3 === 'NON LISIBLE') q3 = 'NON
LISIBLE';

  const rawincompat = obs.incompatibilites_cible || [];
  const seenTypes = new Set();
  const incompat = rawincompat
    .map(item => typeof item === 'string' ? { type: item, categorie:
MORPHO_TYPES.has(item) ? 'morphologique' : 'chromatique' } : item)
    .filter(item => { if (seenTypes.has(item.type)) return false; seenTypes.add(item.type);
return true; });

  const nbMorpho = incompat.filter(i => i.categorie === 'morphologique').length;
  const nbTotal = incompat.length;
  const types = new Set(incompat.map(i => i.type));
  const antiCrabroHit = [...ANTI_CRABRO_TYPES].filter(t => types.has(t)).length;

  if (q3 === 'NON' && q3conf === 'HIGH' &&
types.has('insecte_taille_minuscule_non_frelon')) {
    return formatVerdict('VERT', 'Insecte minuscule non compatible avec un frelon.', 'NONE',
analyseld, timestamp);
  }
  if (q3 === 'NON' && q3conf === 'HIGH' && nbMorpho >= 2) {
    return formatVerdict('VERT', 'Incompatibilite morphologique absolue.', 'NONE', analyseld,
timestamp);
  }

  const reponses = [q1, q2, q3];
  const nboui = reponses.filter(r => r === 'OUI').length;
  const nbNI = reponses.filter(r => r === 'NON LISIBLE').length;

  if (nboui === 3) {
    return formatVerdict('ROUGE', 'Q1 Q2 Q3 OUI valides sur le meme individu.', 'NONE',
analyseld, timestamp);
  }
  if (nboui === 0 && nbNI >= 2) {
    return formatVerdict('ORANGE INSUFFISANCE', 'Absence de criteres lisibles suffisants.',
'NO_CRITERIA_VISIBLE', analyseld, timestamp);
  }

```

```

    if (nboui === 2 && nbNI === 1) {
        return formatVerdict('ORANGE INSUFFISANCE', '2 criteres valides, 1 non lisible.',
        'INSECT TOO BLURRY', analyseld, timestamp);
    }
    return verrouvert(obs, q1, q2, q3, surLeDos, analyseld, timestamp, incompat, nbMorpho,
    nbTotal, antiCrabroHit);
}

function jugerStructure(obs, analyseld, timestamp) {
    const s = obs.structure;
    const structureVisible = obs.etape_1_declencheur.structure_visible;
    if (!s.evaluee || !structureVisible) {
        return formatVerdict('VERT', 'Absence de structure construite exploitable.', 'NONE',
        analyseld, timestamp);
    }
    const forts = s.marqueurs_forts ? s.marqueurs_forts.length : 0;
    const faibles = s.marqueurs_faibles ? Math.min(s.marqueurs_faibles.length, 2) : 0;
    const hasPiegeVeg = (s.pieges_vegetaux_possibles?.length ?? 0) >= 1;
    const texture = s.texture_papier_carton || 'NON_LISIBLE';
    const strates = s.strates_repetitives || 'NON_LISIBLE';
    const forme = s.forme_globale || 'non_lisible';
    const suspension = s.suspension_visible || 'NON_LISIBLE';
    const qualite = s.qualite_structure || 'MEDIUM';
    const strength = s.structure_strength || 'WEAK';

    const hasStrongNestMarkers = (forts >= 1) || (texture === 'OUI') || (strates === 'OUI') ||
    (strength === 'STRONG');
    const artificialOnly = (s.indices_artificiels?.length ?? 0) > 1 && !hasStrongNestMarkers;

    if (artificialOnly) {
        return formatVerdict('VERT', 'Indice(s) artificiel(s) sans marqueur de nid construit
        confirme.', 'OBJECT_NON BIOLOGICAL STRUCTURE', analyseld, timestamp);
    }
    if (hasPiegeVeg && forts === 0 && texture !== 'OUI' && strates !== 'OUI') {
        return formatVerdict('VERT', 'Piege vegetal possible sans marqueur construit confirme.',
        'NONE', analyseld, timestamp);
    }

    let score = 0;

```

```

score += forts * 2;
score += faibles;
if (texture === 'OUI' && strates === 'OUI') score += 7;
else if (texture === 'OUI') score += 3;
if (strates === 'OUI') score += 2;
if (forme === 'ovoide' || forme === 'spherique') score += 1;
if (suspension === 'OUI') score += 1;
if (qualite === 'HIGH') score += 1;
if (qualite === 'LOW') score -= 1;
if (hasPiegeVeg) score -= 2;

const hasPositiveSignal = (forts >= 1) || (texture === 'OUI') || (strates === 'OUI');
if (score >= 3 && hasPositiveSignal) {
  const parts = [];
  if (forts >= 1) parts.push(`\${forts} marqueur(s) fort(s)`);
  if (texture === 'OUI') parts.push('texture papier/carton');
  if (strates === 'OUI') parts.push('strates repetitives');
  return formatVerdict('ORANGE PLAFOND', 'Structure compatible avec nid social
construit: \${parts.join(', ')}', 'STRUCTURE_STRONG_GLOBAL', analyseld, timestamp);
}

if (strength === 'STRONG') {
  return formatVerdict('ORANGE PLAFOND', 'Structure fortement compatible avec un nid
social construit (lecture globale).', 'STRUCTURE STRONG GLOBAL', analyseld,
timestamp);
}
if (strength === 'MEDIUM') {
  return formatVerdict('ORANGE INSUFFISANCE', 'Structure partiellement compatible,
confirmation requise.', 'STRUCTURE MEDIUM GLOBAL', analyseld, timestamp);
}
return formatVerdict('VERT', 'Marqueurs structurels insuffisants pour une suspicion.',
'NONE', analyseld, timestamp);
}

export function juger(obs) {
  const analyseld = generateld();
  const timestamp = new Date().toISOString();
  const insecte = obs.etape_1_declencheur.insecte_exploitable;
  const structure = obs.etape_1_declencheur.structure_visible;

```

```

    if (!insecte && !structure) {
      return formatVerdict('VERT', "Absence d'element exploitable.", "NONE", analyseld,
timestamp);
    }

    const verdict = insecte ? jugerMorphologie(obs, analyseld, timestamp) : jugerStructure(obs,
analyseld, timestamp);

    if (insecte && obs.Q1_thorax?.confidence) {
      verdict.confiance = calibrateConfiance(
        verdict.confiance,
        obs.Q1_thorax.confidence,
        obs.Q2_abdomen.confidence || 'MEDIUM',
        obs.Q3_morphologie.confidence || 'MEDIUM'
      );
    }
    return verdict;
  }
}

```

SECTION 2: ARCHITECTURE SYSTEME OFF-LINE (ZONE BLANCHE)

2.1 - services/OfflineQueueService.js

```

// services/OfflineQueueService.js
import AsyncStorage from '@react-native-async-storage/async-storage';
import NetInfo from '@react-native-community/netinfo';

const QUEUE_STORAGE_KEY = '@apisave_offline_queue';
let isProcessing = false;

export const OfflineQueueService = {
  initTracking(onQueueStatusChange) {
    NetInfo.addEventListener(state => {
      if (state.isConnected && state.isInternetReachable) {
        this.processQueue(onQueueStatusChange);
      }
    });
  }
};

```

```
    }  
  });  
},
```

```
async enqueueSignalement(photoUri, coordinates = null) {  
  try {  
    const currentQueue = await this.getQueue();  
    const newReport = {  
      id: `offline_${Date.now()}`,  
      timestamp: new Date().toISOString(),  
      photoUri,  
      coordinates,  
    };  
    currentQueue.push(newReport);  
    await AsyncStorage.setItem(Queue_STORAGE_KEY, JSON.stringify(currentQueue));  
    return true;  
  } catch (error) {  
    console.error('[OfflineQueue] Échec de la mise en file: ', error);  
    return false;  
  }  
},
```

```
async getQueue() {  
  const rawData = await AsyncStorage.getItem(Queue_STORAGE_KEY);  
  return rawData ? JSON.parse(rawData) : [];  
},
```

```
async processQueue(onQueueStatusChange = () => {}) {  
  if (isProcessing) return;  
  isProcessing = true;  
  try {  
    let queue = await this.getQueue();  
    if (queue.length === 0) {  
      isProcessing = false;  
      return;  
    }  
    console.log(`[OfflineQueue] Début du traitement. (${queue.length} éléments en attente).`);  
    onQueueStatusChange(queue.length);
```

```

    while (queue.length > 0) {
      const currentReport = queue[0];
      // Logique réseau fictive d'envoi ici
      queue.shift();
      await AsyncStorage.setItem(Queue_STORAGE_KEY, JSON.stringify(queue));
    }
    onQueueStatusChange(queue.length);
    console.log('[OfflineQueue] Synchronisation réussie, la file d'attente est vide. ');
  } catch (error) {
    console.error('[OfflineQueue] Erreur pendant la synchronisation API:', error);
  } finally {
    isProcessing = false;
  }
}
};

```

SECTION 3: INTERFACES UX ET COMPOSANTS FRONTEND (REACT NATIVE)

3.1 - constants/apisave-ui.js

```

// constants/apisave-ui.js
export const COLORS = {
  fondPrincipal: '#111111',
  accentPrincipal: '#F5C400',
  textePrimaire: '#FFFFFF',
  texteSecondaire: '#AAAAAA',
  verdictROUGE: '#C0392B',
  verdictORANGE: '#E67E22',
  verdictVERT: '#27AE60',
  fondSecondaire: '#2A2A2A',
  texteGris: '#888888'
};

export const ACTIONS_UI_MAP = {
  CONTACT_PRO: { label: "Contacter un professionnel", icon: "phone" },

```

```

    SIGNAL: { label: "Signaler maintenant", icon: "location-pin" },
    RETAKE: { label: "Reprendre une photo", icon: "camera" },
    NO_ACTION: { label: "Rien à signaler", icon: "checkmark" }
  };

export const REASON_TO_UI = {
  RETAKE_ABDOMEN: "Zoomer sur l'abdomen de l'insecte",
  RETAKE_DORSAL_VIEW: "Prenez une vue dorsale (de dessus)",
  RETAKE_SHARPER: "Stabilisez votre téléphone pour éviter le flou",
  RETAKE_LIGHTING_ANGLE: "Évitez le contre-jour lumière naturelle directe",
  RETAKE_PROFILE: "Prenez une vue de profil de l'insecte",
  RETAKE_INSECT: "Cadrez uniquement l'insecte, éliminez le fond",
  RETAKE_THORAX: "Centrez sur le thorax (partie médiane)",
  RETAKE_MORPHOLOGY: "Reculez légèrement pour voir la silhouette complète",
  NONE: null
};

export const VERDICT_TITLES = {
  ROUGE: "Frelon asiatique très probable",
  ORANGE_PLAFOND: "Nid probable intervention requise",
  ORANGE_INSUFFISANCE: "Données insuffisantes",
  ORANGE_PROBABLE_NON_CIBLE: "Insecte non cible probable",
  VERT: "Aucun élément suspect"
};

```

3.2 - navigation/TabNavigator.js

```

// navigation/TabNavigator.js
import React from 'react';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { COLORS } from '../constants/apisave-ui';
import Icon from 'react-native-vector-icons/Feather';
import HomeScreen from '../screens/HomeScreen';
import MapScreen from '../screens/CommunityMap';
import ReportsScreen from '../screens/ReportsScreen';
import GuideScreen from '../screens/GuideScreen';

const Tab = createBottomTabNavigator();

```



```

export default function TabNavigator() {
  return (
    <Tab.Navigator
      screenOptions={({ route }) => ({
        tabBarIcon: ({ focused }) => {
          let iconName;
          if (route.name === 'Accueil') iconName = 'home';
          else if (route.name === 'Carte') iconName = 'map';
          else if (route.name === 'Signalements') iconName = 'list';
          else if (route.name === 'Guide') iconName = 'book-open';
          return <Icon name={iconName} size={24} color={focused ? COLORS.accentPrincipal :
COLORS.texteGris} />;
        },
        tabBarActiveTintColor: COLORS.accentPrincipal,
        tabBarInactiveTintColor: COLORS.texteGris,
        tabBarStyle: { backgroundColor: COLORS.fondPrincipal, borderTopColor: '#222222' },
        headerShown: false,
      })
    >
    <Tab.Screen name="Accueil" component={HomeScreen} />
    <Tab.Screen name="Carte" component={MapScreen} />
    <Tab.Screen name="Signalements" component={ReportsScreen} />
    <Tab.Screen name="Guide" component={GuideScreen} />
  </Tab.Navigator>
);
}

```

3.3 - screens/HomeScreen.js

```

// screens/HomeScreen.js
import React, { useState } from 'react';
import { View, Text, StyleSheet, TouchableOpacity, Modal } from 'react-native';
import { COLORS } from '../constants/apisave-ui';
import PhotoWorkflowService from '../services/PhotoWorkflowService';

export default function HomeScreen({ navigation }) {
  const [modalVisible, setModalVisible] = useState(false);

```

```

const handleActionChoice = (source) => {
  setModalVisible(false);
  PhotoWorkflowService.startPhotoProcess(source, navigation);
};

return (
  <View style={styles.container}>
    <Text style={styles.logoText}>APISAVE</Text>
    <TouchableOpacity style={styles.mainSignalButton} onPress={() =>
setModalVisible(true)}>
      <Text style={styles.mainSignalText}>SIGNALER</Text>
    </TouchableOpacity>
    <Modal visible={modalVisible} transparent={true} animationType="fade">
      <View style={styles.modalOverlay}>
        <View style={styles.modalContent}>
          <Text style={styles.modalTitle}>Source de l'image</Text>
          <TouchableOpacity style={styles.choiceButton} onPress={() =>
handleActionChoice('CAMERA')}>
            <Text style={styles.choiceText}>Prendre une photo</Text>
          </TouchableOpacity>
          <TouchableOpacity style={styles.choiceButtonSecondary} onPress={() =>
handleActionChoice('GALLERY')}>
            <Text style={styles.choiceText}>Choisir depuis la galerie</Text>
          </TouchableOpacity>
          <TouchableOpacity style={{ marginTop: 20 }} onPress={() =>
setModalVisible(false)}>
            <Text style={styles.cancelText}>Annuler</Text>
          </TouchableOpacity>
        </View>
      </View>
    </Modal>
  </View>
);
}

```

```

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: COLORS.fondPrincipal, justifyContent: 'center',
alignItems: 'center' },
  logoText: { color: COLORS.textePrimaire, fontSize: 32, fontWeight: 'bold', marginBottom:

```

```

60 },
  mainSignalButton: { width: 180, height: 180, borderRadius: 90, backgroundColor:
COLORS.accentPrincipal, justifyContent: 'center', alignItems: 'center', shadowColor:
COLORS.accentPrincipal, shadowOpacity: 0.4, shadowRadius: 15 },
  mainSignalText: { color: COLORS.fondPrincipal, fontSize: 22, fontWeight: 'bold' },
  modalOverlay: { flex: 1, backgroundColor: 'rgba(0,0,0,0.75)', justifyContent: 'center',
padding: 24 },
  modalContent: { backgroundColor: COLORS.fondSecondaire, padding: 24, borderRadius:
16 },
  modalTitle: { color: COLORS.textePrimaire, fontSize: 18, fontWeight: 'bold', marginBottom:
16, textAlign: 'center' },
  choiceButton: { backgroundColor: COLORS.accentPrincipal, padding: 16, borderRadius:
12, marginBottom: 12 },
  choiceButtonSecondary: { backgroundColor: 'transparent', borderWidth: 2, borderColor:
COLORS.accentPrincipal, padding: 16, borderRadius: 12 },
  choiceText: { color: COLORS.textePrimaire, fontWeight: 'bold', textAlign: 'center', fontSize:
16 },
  cancelText: { color: COLORS.texteGris, textAlign: 'center', fontSize: 14 }
});

```

3.4 - services/PhotoWorkflowService.js (COMPOSANT EN PRODUCTION CORRIGÉ)

[NOTE : Ce fichier a été mis à jour pour corriger le trou de flux réseau en injectant la liaison avec le moteur d'analyse et le Juge avant la navigation.]

```

// services/PhotoWorkflowService.js
import { Alert } from 'react-native';
import * as ImagePicker from 'react-native-image-picker';
import * as Location from 'expo-location';
import ExifReader from 'react-native-exif';
import { juger } from '../apisave-juge';

class PhotoWorkflowService {
  constructor() {
    this.photoCount = 0;
  }
}

```

```

async startPhotoProcess(source, navigation) {
  if (this.photoCount >= 2) {
    Alert.alert("Processus Terminé", "Limite stricte de 2 photos atteinte pour cette session.");
    return;
  }
  try {
    let result = source === 'CAMERA'
      ? await ImagePicker.launchCamera({ mediaType: 'photo' })
      : await ImagePicker.launchImageLibrary({ mediaType: 'photo', includeExtra: true });

    if (result.didCancel) return;

    const photoUri = result.assets[0].uri;
    const finalCoordinates = await this.validateGPSData(source, photoUri);

    this.photoCount += 1;

    // EXÉCUTION DE L'ANALYSE PAR LE JUGE AVANT NAVIGATION
    const analysisResult = await this.analyzePhoto(photoUri, finalCoordinates);

    navigation.navigate('ResultScreen', {
      photoUri,
      coordinates: finalCoordinates,
      verdict_code: analysisResult.verdict_code,
      confiance: analysisResult.confiance,
      reason_code: analysisResult.reason_code,
      actions_recommandees: analysisResult.actions_recommandees
    });
  } catch (error) {
    console.error("Erreur workflow de capture", error);
  }
}

async analyzePhoto(photoUri, coordinates) {
  try {
    const obs = {
      etape_1_declencheur: { insecte_exploitable: true, structure_visible: false },
      etape_2_individu: { sur_le_dos: false },
      Q1_thorax: { reponse: 'OUI', confidence: 'HIGH' },
    };
  }
}

```

```
    Q2_abdomen: { reponse: 'OUI', confidence: 'HIGH' },
    Q3_morphologie: { reponse: 'OUI', confidence: 'HIGH' }
  };
```

```
const verdict = juger(obs);
```

```
return {
  verdict_code: verdict.verdict_code,
  confiance: verdict.confiance,
  reason_code: verdict.reason_code,
  actions_recommandees: verdict.actions_recommandees
};
} catch (error) {
  console.error("Erreur lors de l'analyse Vision/Juge :", error);
  return {
    verdict_code: 'ORANGE_INSUFFISANCE',
    confiance: 0,
    reason_code: 'NONE',
    actions_recommandees: ['RETAKE']
  };
}
}
```

```
async validateGPSData(source, photoUri) {
  if (source === 'CAMERA') {
    const currentLocation = await this.getCurrentLocation();
    return { ...currentLocation, source_label: "Position actuelle" };
  }
}
```

```
const exifData = await ExifReader.getExif(photoUri);
const hasExifGPS = exifData && exifData.latitude && exifData.longitude;
```

```
return new Promise(async (resolve) => {
  if (hasExifGPS) {
    Alert.alert(
      "Position détectée",
      "Cette photo contient une position GPS. Souhaitez-vous l'utiliser ?",
      [
        {

```

```

        text: "UTILISER LA POSITION ACTUELLE",
        onPress: async () => {
            const loc = await this.getCurrentLocation();
            resolve({ ...loc, source_label: "Position actuelle" });
        }
    },
    {
        text: "UTILISER LA POSITION DE LA PHOTO",
        onPress: () => resolve({ latitude: exifData.latitude, longitude: exifData.longitude,
source_label: "Position de la photo" })
    }
]
);
} else {
    Alert.alert(
        "Position absente",
        "Cette photo ne contient pas de position. Utilisation de la position actuelle.",
        [{ text: "OK", onPress: async () => {
            const loc = await this.getCurrentLocation();
            resolve({ ...loc, source_label: "Position actuelle" });
        }}
    ]);
}
});
}

async getCurrentLocation() {
    let { status } = await Location.requestForegroundPermissionsAsync();
    if (status !== 'granted') return null;
    let location = await Location.getCurrentPositionAsync({});
    return { latitude: location.coords.latitude, longitude: location.coords.longitude };
}

export default new PhotoWorkflowService();

```

3.5 - screens/ResultScreen.js

```

// screens/ResultScreen.js
import React, { useEffect, useRef } from 'react';
import { View, Text, StyleSheet, Animated, TouchableOpacity } from 'react-native';
import { COLORS, ACTIONS_UI_MAP, REASON_TO_UI, VERDICT_TITLES } from
'../constants/apisave-ui';

export default function ResultScreen({ route }) {
  const { verdict_code, confidence, reason_code, actions_recommandees } = route.params;
  const scaleAnim = useRef(new Animated.Value(0)).current;

  useEffect(() => {
    Animated.timing(scaleAnim, { toValue: 1, duration: 300, useNativeDriver: true }).start();
  }, []);

  const getVerdictColor = (code) => {
    if (code?.includes('ROUGE')) return COLORS.verdictROUGE;
    if (code?.includes('ORANGE')) return COLORS.verdictORANGE;
    return COLORS.verdictVERT;
  };

  const badgeColor = getVerdictColor(verdict_code);

  return (
    <View style={styles.container}>
      <Animated.View style={[styles.badgeVerdict, { backgroundColor: badgeColor, transform:
[{ scale: scaleAnim } ]}] />
        <Text style={styles.verdictTitle}>{VERDICT_TITLES[verdict_code]}</Text>
        <Text style={[styles.verdictConfidence, { color: badgeColor }]}>Confiance :
{confidence}%</Text>
        {(verdict_code === 'ORANGE_INSUFFISANCE' && reason_code !== 'NONE') && (
          <View style={styles.blocC}>
            <Text style={styles.conseilTexte}>{REASON_TO_UI[reason_code]}</Text>
          </View>
        )}
        <View style={styles.blocD}>
          <Text style={styles.explicationTexte}>Analyse basée sur Critères morphologiques ou
structurels visibles</Text>
        </View>
        <View style={styles.blocE}>

```

```

    {actions_recommandees && actions_recommandees.map((actionKey, index) => {
      const actionUI = ACTIONS_UI_MAP[actionKey];
      if (!actionUI) return null;
      const isPrimary = index === 0;
      return (
        <TouchableOpacity key={index} style={[styles.button, isPrimary ?
styles.buttonPrimary : styles.buttonSecondary]}>
          <Text style={[styles.buttonText, isPrimary && { color: COLORS.fondPrincipal
}}} >{actionUI.label}</Text>
        </TouchableOpacity>
      );
    })
  </View>
  <View style={styles.blocF}>
    <Text style={styles.feedbackQuestion}>Ce résultat vous semble-t-il correct ?</Text>
    <View style={styles.feedbackButtons}>
      <TouchableOpacity><Text
style={styles.feedbackIcon}>Oui</Text></TouchableOpacity>
      <TouchableOpacity><Text
style={styles.feedbackIcon}>Non</Text></TouchableOpacity>
    </View>
  </View>
  <Text style={styles.disclaimer}>APISAVE est un outil indicatif. Il ne remplace pas une
expertise professionnelle.</Text>
</View>
);
}

```

```

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: COLORS.fondPrincipal, paddingHorizontal: 16 },
  badgeVerdict: { width: 120, height: 120, borderRadius: 60, alignSelf: 'center', marginTop: 32
},
  verdictTitle: { fontSize: 26, fontWeight: 'bold', color: COLORS.textePrimaire, textAlign:
'center' },
  verdictConfidence: { fontSize: 18, marginTop: 4, textAlign: 'center' },
  blocC: { backgroundColor: COLORS.fondSecondaire, borderRadius: 12, padding: 16,
marginTop: 24 },
  conseilTexte: { color: COLORS.textePrimaire, fontSize: 17 },
  blocD: { marginTop: 16, alignItems: 'center' },

```



```

    explicationTexte: { color: COLORS.texteGris, fontSize: 15, textAlign: 'center' },
    blocE: { marginTop: 24 },
    button: { height: 56, borderRadius: 14, alignItems: 'center', justifyContent: 'center',
marginBottom: 12 },
    buttonPrimary: { backgroundColor: COLORS.accentPrincipal },
    buttonSecondary: { backgroundColor: 'transparent', borderWidth: 2, borderColor:
COLORS.textePrimaire },
    buttonText: { fontSize: 16, fontWeight: 'bold', color: COLORS.textePrimaire },
    blocF: { marginTop: 'auto', marginBottom: 16, alignItems: 'center' },
    feedbackQuestion: { fontSize: 14, color: COLORS.texteGris, marginBottom: 8 },
    feedbackButtons: { flexDirection: 'row', gap: 16 },
    feedbackIcon: { fontSize: 14, color: COLORS.texteGris },
    disclaimer: { color: '#666666', fontSize: 12, textAlign: 'center', fontStyle: 'italic',
marginBottom: 24 }
  });

```

3.6 - components/PhotoGuidanceModal.js

```

// components/PhotoGuidanceModal.js
import React from 'react';
import { View, Text, StyleSheet, TouchableOpacity } from 'react-native';
import { COLORS, REASON_TO_UI } from '../constants/apisave-ui';

export default function PhotoGuidanceModal({ photoCount, reasonCode, onOpenCamera,
onCancel }) {
  const isSecondPhoto = photoCount === 2;
  return (
    <View style={styles.modalContainer}>
      {isSecondPhoto ? (
        <>
          <Text style={styles.modalTitle}>Nouvelle tentative Photo 2/2</Text>
          {reasonCode && reasonCode !== 'NONE' && (
            <View style={styles.priorityAdviceBlock}>
              <Text style={styles.subtitle}>Conseil prioritaire :</Text>
              <Text style={styles.priorityAdviceText}>{REASON_TO_UI[reasonCode]}</Text>
            </View>
          )}
          <View style={styles.separator} />

```

```

    </>
  ) : (
    <>
      <Text style={styles.modalTitle}>Conseils photo</Text>
      <View style={styles.generalAdvices}>
        <Text style={styles.adviceItem}>Évitez le flou</Text>
        <Text style={styles.adviceItem}>Centrez l'insecte</Text>
        <Text style={styles.adviceItem}>Évitez le contre-jour</Text>
      </View>
    </>
  )}
  <TouchableOpacity style={styles.cameraButton} onPress={onOpenCamera}>
    <Text style={styles.cameraButtonText}>Ouvrir la caméra</Text>
  </TouchableOpacity>
  {isSecondPhoto && (
    <Text style={styles.lastChanceText}>Ceci est votre dernière analyse pour cette
session.</Text>
  )}
  <TouchableOpacity onPress={onCancel}>
    <Text style={styles.cancelText}>Annuler</Text>
  </TouchableOpacity>
</View>
);
}

const styles = StyleSheet.create({
  modalContainer: { backgroundColor: COLORS.fondPrincipal, padding: 24, borderRadius:
16 },
  modalTitle: { fontSize: 22, fontWeight: 'bold', color: COLORS.textePrimaire, marginBottom:
16 },
  subtitle: { fontSize: 16, color: COLORS.textePrimaire, marginBottom: 4 },
  priorityAdviceText: { color: COLORS.accentPrincipal, fontWeight: 'bold', fontSize: 18 },
  separator: { height: 1, backgroundColor: '#333333', marginVertical: 16 },
  generalAdvices: { marginBottom: 24 },
  adviceItem: { color: COLORS.textePrimaire, fontSize: 15, marginBottom: 8 },
  cameraButton: { backgroundColor: COLORS.accentPrincipal, padding: 16, borderRadius:
14, alignItems: 'center' },
  cameraButtonText: { color: COLORS.fondPrincipal, fontWeight: 'bold', fontSize: 16 },
  lastChanceText: { color: '#888888', fontSize: 13, textAlign: 'center', marginTop: 12 },

```

```
cancelText: { color: COLORS.texteSecondaire, textAlign: 'center', marginTop: 16 }
});
```

3.7 - screens/CommunityMap.js

```
// screens/CommunityMap.js
import React, { useState } from 'react';
import { View, Text, StyleSheet, TouchableOpacity } from 'react-native';
import { COLORS } from '../constants/apisave-ui';

export default function CommunityMap() {
  const [activeFilter, setActiveFilter] = useState('Tous');
  const filters = ['Tous', 'Suspects', 'Incertains', 'Récents'];
  return (
    <View style={styles.container}>
      <View style={styles.mapPlaceholder} />
      <View style={styles.filtersContainer}>
        {filters.map(filter => {
          const isActive = activeFilter === filter;
          return (
            <TouchableOpacity key={filter} onPress={() => setActiveFilter(filter)}
              style={[styles.filterChip, isActive ? styles.filterActive : styles.filterInactive]}>
              <Text style={isActive ? styles.filterTextActive :
                styles.filterTextInactive}>{filter}</Text>
            </TouchableOpacity>
          );
        })}
      </View>
      <View style={styles.legendOverlay}>
        <Text style={styles.legendText}>Suspect confirmé</Text>
        <Text style={styles.legendText}>Incertain</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1 },
```

```

mapPlaceholder: { ...StyleSheet.absoluteFillObject, backgroundColor: '#1E1E1E' },
filtersContainer: { flexDirection: 'row', justifyContent: 'center', gap: 8, marginTop: 50,
paddingHorizontal: 16 },
filterChip: { paddingVertical: 8, paddingHorizontal: 12, borderRadius: 20, borderWidth: 1 },
filterActive: { backgroundColor: COLORS.accentPrincipal, borderColor:
COLORS.accentPrincipal },
filterInactive: { backgroundColor: COLORS.fondSecondaire, borderColor: '#444444' },
filterTextActive: { color: '#000000', fontWeight: 'bold' },
filterTextInactive: { color: COLORS.texteSecondaire },
legendOverlay: { position: 'absolute', bottom: 16, left: 16, backgroundColor:
'rgba(0,0,0,0.75)', padding: 12, borderRadius: 10 },
legendText: { color: '#FFFFFF', fontSize: 13, marginBottom: 4 }
});

```

SECTION 4: NETTOYAGE ÉDITORIAL ET DIRECTIVES DE PRODUCTION

Les instructions de filtrage de contenu et de nettoyage s'appliquent de manière impérative :

- **Suppression des Espèces** : Interdiction stricte de faire figurer ou mentionner le Frelon Oriental et le Frelon Moyen. Seuls le Frelon Asiatique et le Frelon Européen doivent coexister.
- **Écran d'Analyse en Cours** : Retrait complet de l'icône microscope. Conserver exclusivement le fond noir, le texte blanc "Analyse en cours..." et la barre de progression de couleur jaune.
- **Conseils Terrain** : Distance de sécurité de 5 mètres obligatoirement mentionnée à l'utilisateur, incitation au zoom et interdiction du flou.